数组数据特点： 全都是有序地排列，按照一定顺序排列，怎么放进去的就已经固定了；
//数组缺点：取值的时候是通过下标取值，不能体现一条数据的特性和作用；
//Json对象：目的：用来保留数据，用于后期获取数据；
//Json对象：是存储在一个花括号里面 ；

var obj = {}; //当对象里面没有值的时候， 是一个空对象；
/* 说明：
1. 也是一种储存对象的结构，并且这个结构通常在与后台交互的时候使用,如注册、登陆；
2. 用法：通常用来做数据交互的，前端数据传输到后面都是通过Json来传输的，以对象的格式来传输。 */

//如何来存储值：通过键值对的方式来存储，
键：属性;值：属性值；
var obj = { name : 'candy' , age : 18 , length : 18 style:{ background : 'red'; } }

/* 与数组的优点：
当前这条数据相比数组是可以把属性名给反应出来; */

/* 注意：
1. 属性值可以为任意值。 2. 与数组的区别：数组使用下标，json对象使用属性; */
in 检测json对象中是否有某个属性
attribute in json

检测：in(关键字)：用来判断某个对象是否存在某条属性;

```
if( 'name' in obj ){  //注意：这里的属性要使用字符串引起来，不然会被当成变量。
    console.log( '存在' );
}else{
    console.log( '不存在' );
}
//不管为真值还是为假值，只要在对象里面，就能检测到。
```

如何给空对象添加属性：

var obj = {}; obj.name = 'candy'; //如果原先有值的话，就是访问。如果通过等号就是赋值的过程;

console.log(obj.name); // ->输出 candy ;

//如果原先里面有值的话，会被覆盖;

对象的创建方式：

```
var obj = {};    //  字面量 或 直接量;

var obj1 = new Object() ; //通过底层的构造函数Obj加new来执行;必须用关键字来执行。console.log( obj1 )  输出 -> {} ;里面没有属性。obj1.name = 'candy';

var obj2 = Object.create( {} );
//这种方式也能创建一个对象，过程会复杂些。这里是依照一个对象来创建另外一个对象;这里生成对象的时候，有一些选项。比如你这条属性可否被遍历、被循环、被删除、被修改，
```

删除对象

delete obj.name;

//可以用中括号[]来代替点操作.

console.log(obj['age']);

定义：

除了在JS中用来记录数据外，还可以与后台进行数据交互，这个时候我们通常把这个对象叫做Json对象。 如果不是用来与后台进行交互，叫做对象也是可以的。

/* 注意：

1.如果作为Json与后台进行数据交换，所有的数据的属性名跟属性值都必须为字符串。而且为双引号的字符串， 不可以为单引号。 var obj = { name : 'candy' , age : 18 , lenth : 18 } 2.在其它语言中，双引号引起来的都为字

符串，但是单引号的是不一定的。 3.在Js中，不管你用单引号还是双引号都没有什么区别。 4.如果只作用于前端，那么是单引号或双引号都可以。 */

对象的序列化与反序列化

```
//把这个数据作为对象传送到后台，都需要加上引号;一个严格的JSON对象的属性和属性值都应该加上引号的。
var obj = {
  name : 'candy';
  age : ' 18 ';
  sex : ' man ';
};
//以上不可以作为对象传送给后台，因为没有加上双引号;
//如果不确定是否有加上双引号的时候，如果没有加上双引号的时候，那么就无法传送数据。所以需要对象进行数列化与反序列化。

//序列化:  把不标准的Json对象变成标准的Json对象，就是给每个属性加上双引号。
var obj1 = JSON.stringfy( obj );//进行序列化,传一个不标准json对象返回一个新的对象;
```

```
console.log( obj1 );  //标准的字符串的obj1.  {'name':'candy','age':'18','sex':'man'}
console.log( typeof obj1 );  // 是一个字符串;

//序列化的结果:JSON格式的字符串;
console.log( obj1.name );     //已经访问不到对应的结果.

//如何把一个JSON对象的字符串反序列化为一个对象:
var obj2 = JSON.parse(obj1);
console.log( obj2 );    //输出一个对象;
console.log( obj2.name );  // candy .
```

```
/*
    序列化的结果: 返回一个跟obj1一模一样的对象;
    1.在自己身上扩展的时候不会影响原先的样式。
    2.先通过序列化stringify先返回字符串，再通过反序列化的parse把一个字符串返回为对象，这个时候返回一个新的与原先obj一模一样的对象。
    3.当前返回一个新的对象在会内存空间开辟另一块地址来保存对象，而对象指向原先对象。
*/
```

注意：在进行序列化的时候会忽略掉函数，会把函数过滤。

拷贝对象

```
var obj = {}
var obj1 = {}
var obj2 = obj;   //两个指针指向了一块内存地址【对象的拷贝、引用】
console.log( obj === obj1 )//false
console.log( obj2 === obj )//true
```

call apply bind

```
var arr = [1];
function fn(){
    console.log( this );
}
document.onclick = fn;

//改变this的指向;
document.onclick = fn.call( arr );  //会主动执行!
document.onclick = fn.bind( arr );  //bind会主动执行!

//如需要bind主动执行,需要加括号!
fn.bind( arr )();

/*
    1.call( this指向的对象, arg1,arg2 ... )
    2.apply( this指向的对象, [arr1,arr2] ) , apply指向对象必须作为数组的内容;
    3.bind : ( 不会主动执行函数, arr1 ,arr2...)
*/
```

```
var arr = ["666"];
function fn (a,b,c){
    console.log( this )
    console.log( a )
    console.log( b )
    console.log( c )
}
window.fn.call(arr,666,777,888)
window.fn.apply(arr,[666,777,888]);
 // 666 777 888
```

注意:

call中的第一个是改变this的指向的，后面的参数都作为实参进行传递

```
document.onclick = fn.bind(arr)
```

第一个是改变this的指向，第二参数是事件对象

如果bind()里面的值为null指针/undefined，this的指向不会指向null，指向window;